

Distributed and Parallel Computing

II Midterm Exam — Preparation Questions

Lectures 9, 10 & 11: GPU/CUDA · Distributed Fundamentals · CAP, Consensus & Raft

Instructor: Dr. Erekle Magradze | Ilia State University

exactly one correct answer.

1. What kind of problem is best suited to GPU acceleration?
 - A. Sequential control flow with heavy branching
 - B. Data parallelism: the same simple operation applied to millions of independent data points
 - C. Latency-sensitive single-threaded work
 - D. OS-level scheduling tasks
2. Compared to a CPU, an individual GPU core is generally:
 - A. Much stronger but far fewer in number
 - B. Much weaker individually, but present in far greater numbers
 - C. Identical in strength and count
 - D. Stronger and more numerous
3. Roughly how many cores does a modern data-center GPU have?
 - A. 8-64
 - B. Around 1,000
 - C. 10,000+
 - D. Exactly 32
4. Where do CPUs win over GPUs?
 - A. Dense linear algebra
 - B. Sequential, branchy, latency-bound work
 - C. Neural network training
 - D. Particle simulations
5. A CPU invests silicon mainly in:
 - A. Thousands of tiny arithmetic units
 - B. Branch predictors, large caches, and deep pipelines
 - C. High-bandwidth memory only
 - D. Tensor cores
6. How does a GPU hide memory latency?
 - A. With very large per-core caches
 - B. By thread oversubscription — switching to warps that have work ready
 - C. By running at much higher clock speeds
 - D. By avoiding global memory entirely

7. What does SIMT stand for?
- A. Single Instruction, Multiple Threads
 - B. Sequential Instruction, Multiple Tasks
 - C. Single Instruction, Multiple Transfers
 - D. Shared Instruction, Multiple Threads
8. In CUDA, a kernel describes:
- A. What all threads collectively do in sequence
 - B. What ONE thread should do
 - C. Only host-side memory management
 - D. The grid launch configuration
9. Threads on a GPU are grouped into warps of:
- A. 8
 - B. 16
 - C. 32
 - D. 64
10. What is warp divergence?
- A. When warps run on different SMs
 - B. When threads in a warp take different branches and the warp serializes
 - C. When a warp exceeds 32 threads
 - D. When two warps access the same memory
11. How does SIMT differ from classical SIMD?
- A. SIMT forces every lane to execute every step
 - B. SIMT lets each thread have its own program counter, so divergence is allowed (just slow)
 - C. SIMD allows per-thread program counters
 - D. They are identical
12. What is a Streaming Multiprocessor (SM)?
- A. The single global memory controller
 - B. The basic GPU building block containing CUDA cores, schedulers, register files, and fast memory
 - C. A type of CPU core
 - D. The host-side launch queue
13. The warp scheduler's job is to:
- A. Compile kernels at runtime
 - B. Pick which warps to run on a given cycle, hiding latency
 - C. Allocate global memory
 - D. Transfer data between host and device

14. What is the correct ordering of CUDA's thread hierarchy from smallest to largest?
- A. Block -> Thread -> Grid
 - B. Thread -> Block -> Grid
 - C. Grid -> Block -> Thread
 - D. Warp -> Grid -> Block
15. Threads within the same block can:
- A. Synchronize across the entire grid
 - B. Share fast memory and synchronize with each other
 - C. Never communicate
 - D. Only communicate via global memory
16. Within a single kernel launch, blocks in a grid:
- A. Run in strict sequential order
 - B. Run independently, with no inter-block synchronization
 - C. Always share registers
 - D. Must all reside on one SM
17. Which GPU memory is fastest and explicitly controlled by the programmer?
- A. Global memory
 - B. Constant memory
 - C. Shared memory
 - D. L2 cache
18. Registers in the GPU memory hierarchy are scoped to:
- A. The whole grid
 - B. A single block
 - C. A single thread
 - D. The entire chip
19. Global memory is characterized as:
- A. The fastest and smallest memory
 - B. The slowest and biggest memory, where host transfers land
 - C. Per-thread scratch space
 - D. Read-only and broadcast
20. What does CUDA stand for?
- A. Compute Unified Device Architecture
 - B. Concurrent Unified Data Access
 - C. CUDA Universal Device API
 - D. Compute Unit Distributed Array

21. CUDA works on:
- A. Any GPU vendor
 - B. Only NVIDIA hardware
 - C. Only AMD hardware
 - D. CPUs only
22. Which qualifier marks a function as a CUDA kernel?
- A. `__device__`
 - B. `__host__`
 - C. `__global__`
 - D. `__shared__`
23. What does `nvcc` do with a `.cu` file?
- A. Runs it directly without compilation
 - B. Splits it into host code (system C++ compiler) and device code (PTX then GPU machine code)
 - C. Only compiles the host portion
 - D. Converts it to Python
24. In the launch `hello<<<1, 10>>>()`, the values mean:
- A. 10 blocks of 1 thread
 - B. 1 block of 10 threads
 - C. 10 grids
 - D. 1 thread repeated 10 times
25. Why is `cudaDeviceSynchronize()` needed after a kernel launch?
- A. To allocate device memory
 - B. Kernel launches are asynchronous; it blocks until the GPU finishes
 - C. To free device memory
 - D. To copy data to the host
26. The standard expression for a thread's global 1D index is:
- A. `threadIdx.x` only
 - B. `blockIdx.x * blockDim.x + threadIdx.x`
 - C. `blockDim.x + gridDim.x`
 - D. `gridDim.x * threadIdx.x`
27. Why is a bounds check (`if (i < n)`) important in kernels?
- A. It speeds up arithmetic
 - B. Ceiling division may launch more threads than data elements, so extra threads must not access out of bounds
 - C. It synchronizes the block
 - D. It allocates memory

28. The ceiling-division block count $(N + \text{threads} - 1) / \text{threads}$ guarantees:
- A. Exactly one block
 - B. Enough threads are launched to cover N
 - C. Threads never exceed N
 - D. Shared memory is allocated
29. Host memory and device memory are:
- A. The same address space
 - B. Separate address spaces
 - C. Always automatically synchronized
 - D. Both managed only by the kernel
30. What happens if you pass a host pointer to a kernel?
- A. It works correctly
 - B. It compiles fine but crashes or produces garbage at runtime
 - C. It is automatically copied to the device
 - D. The compiler rejects it
31. What does a grid-stride loop accomplish?
- A. It forces one thread per element
 - B. It decouples the work size N from the launch configuration; each thread handles multiple items
 - C. It eliminates the need for global memory
 - D. It synchronizes across blocks
32. In a grid-stride loop, the stride is computed as:
- A. `blockDim.x` only
 - B. `blockDim.x * gridDim.x`
 - C. `gridDim.x` only
 - D. `threadIdx.x * blockDim.x`
33. When is shared memory most beneficial?
- A. When each thread reads unique global data once
 - B. When several threads in the same block read the SAME global data
 - C. When no synchronization is needed
 - D. When data is write-only
34. `__syncthreads()` synchronizes:
- A. All threads across the grid
 - B. All threads within one block
 - C. Only threads in a warp
 - D. Host and device

35. What is the danger of placing `__syncthreads()` inside an if-statement that some threads skip?
- A. Slower memory access
 - B. It deadlocks the block
 - C. It corrupts global memory
 - D. Nothing — it is safe
36. In parallel reduction, what does thread 0 do at the end?
- A. Loads all input values
 - B. Writes the block's partial sum to `block_sums[blockIdx.x]`
 - C. Synchronizes the grid
 - D. Frees device memory
37. PyCUDA's SourceModule:
- A. Compiles CUDA C provided as a string at runtime
 - B. Is a pure-Python kernel implementation
 - C. Requires a separate offline build step
 - D. Only works without a GPU
38. In PyCUDA, block and grid parameters are:
- A. Always 1D integers
 - B. Always 3D tuples, padding unused dims with 1
 - C. Optional and default to the whole GPU
 - D. Specified in bytes
39. PyCUDA's gpuarray provides:
- A. A low-level driver wrapper only
 - B. NumPy-like arrays that live on the GPU, hiding allocation and copying
 - C. A replacement for `nvcc`
 - D. CPU-only computation
40. In PyCUDA, `cuda.In(a)` and `cuda.Out(c)` are used to:
- A. Compile the kernel
 - B. Mark arrays for host->device input and device->host output transfer automatically
 - C. Synchronize the context
 - D. Allocate constant memory
41. Distributed systems break which three single-machine guarantees at once?
- A. One process, one memory, one clock
 - B. One CPU, one disk, one network
 - C. One thread, one lock, one queue
 - D. One user, one node, one socket

- 42.** In a distributed system, the only way for independent nodes to share state is to:
- A.** Use shared memory
 - B.** Send a message
 - C.** Access a global clock
 - D.** Use a single mutex
- 43.** Who catalogued the 8 fallacies of distributed computing?
- A.** Leslie Lamport
 - B.** Peter Deutsch (Sun Microsystems)
 - C.** Eric Brewer
 - D.** Fischer, Lynch, and Paterson
- 44.** Which is the first fallacy of distributed computing?
- A.** Latency is zero
 - B.** The network is reliable
 - C.** Bandwidth is infinite
 - D.** Topology doesn't change
- 45.** The recommended antidote to the 'network is reliable' fallacy is:
- A.** Compression
 - B.** Retries, idempotency, and sequence numbers
 - C.** Encryption
 - D.** Service discovery
- 46.** Typical cross-region network latency is around:
- A.** 0 ms
 - B.** 0.5-1 ms
 - C.** 50-150 ms
 - D.** Several minutes
- 47.** The fix for the 'bandwidth is infinite' fallacy is to:
- A.** Use TLS everywhere
 - B.** Serialise, compress, and avoid huge hot-path payloads
 - C.** Add more administrators
 - D.** Increase timeouts
- 48.** The 'network is secure' fallacy is addressed by:
- A.** Binary protocols
 - B.** TLS everywhere, mutual auth, and input validation
 - C.** Bigger buffers
 - D.** Circuit breakers

49. 'Topology doesn't change' is countered with:
- A. Compression
 - B. Service discovery, dynamic routing, and circuit breakers
 - C. Idempotency
 - D. Atomic clocks
50. The fallacy 'there is one administrator' is fixed by:
- A. Versioned APIs, backward compatibility, and contract tests
 - B. Encryption
 - C. Heartbeats
 - D. Bigger payloads
51. 'Transport cost is zero' is addressed by:
- A. Using JSON everywhere
 - B. Binary protocols like protobuf or Avro over JSON at scale
 - C. Removing retries
 - D. Disabling TLS
52. The 'network is homogeneous' fallacy reminds you to:
- A. Assume all nodes are identical
 - B. Not assume features (IPv4/IPv6, MTUs, firewalls) are available uniformly
 - C. Use one administrator
 - D. Skip serialization
53. The eight fallacies group into which three engineering reflexes?
- A. Expect loss, respect cost, trust nothing
 - B. Read, write, execute
 - C. Crash, omission, Byzantine
 - D. Strong, causal, eventual
54. A crash failure is defined as a node that:
- A. Sends wrong data
 - B. Stops entirely and stays silent, producing no further output
 - C. Loses some messages but keeps running
 - D. Colludes with an adversary
55. Why is crash failure the friendliest case?
- A. It never actually happens
 - B. A crashed node is consistent — it does nothing, never wrong things
 - C. It can be ignored
 - D. It self-repairs

56. Heartbeat-based crash detection works by:
- A. Sending wrong data deliberately
 - B. Treating a node as dead if no heartbeat arrives within a timeout
 - C. Voting among replicas
 - D. Encrypting all traffic
57. An omission failure is when a node:
- A. Stops entirely and never recovers
 - B. Is running, but some messages it sends or receives are lost
 - C. Lies about its state
 - D. Has no clock
58. Why is omission subtler than crash failure?
- A. The node is faster
 - B. The sender can't tell 'message lost' from 'reply slow'
 - C. It never causes partial state
 - D. It is always detectable
59. The cure for omission failures is:
- A. Voting and quorums
 - B. Acknowledgements, retransmission, and dedup
 - C. $3f+1$ replicas
 - D. Atomic clocks
60. Why must a retried request be idempotent?
- A. To make it faster
 - B. Because after a timeout you don't know if the original was processed, so a safe retry must not double-apply
 - C. To compress it
 - D. To encrypt it
61. A Byzantine failure is one where a node:
- A. Simply stops
 - B. Loses a few messages
 - C. Behaves arbitrarily — sends wrong data, lies, or colludes with an adversary
 - D. Has a slow clock
62. Tolerating f Byzantine liars classically requires how many nodes?
- A. $f + 1$
 - B. $2f + 1$
 - C. $3f + 1$
 - D. f^2

63. Which failure model does this course implement, leaving the other named-only?
- A. Implements Byzantine; names crash
 - B. Implements crash and omission; names Byzantine but leaves it to a later course
 - C. Implements all three fully
 - D. Implements none
64. Why can't physical clocks reliably order events across machines?
- A. They are too fast
 - B. Each machine's clock drifts independently and network sync has hard limits
 - C. They use GPS
 - D. They are perfectly synchronized
65. NTP accuracy over the public internet is approximately:
- A. ~10 ms
 - B. ~1 ms
 - C. microsecond-level
 - D. exactly 0 ms
66. Microsecond-level clock accuracy is achievable only with:
- A. Standard NTP
 - B. GPS-disciplined atomic clocks (e.g. Spanner TrueTime)
 - C. Lamport clocks
 - D. Vector clocks
67. Lamport's happens-before relation captures:
- A. Real wall-clock time
 - B. Causality / a partial order of events
 - C. Byzantine behaviour
 - D. Network bandwidth
68. Which is one of Lamport's three happens-before rules?
- A. Receiving a message happens-before sending it
 - B. Sending a message happens-before receiving it
 - C. Concurrent events are always ordered
 - D. Clocks must be synchronized
69. On receiving a message, a Lamport clock updates its counter to:
- A. $C = C + 1$
 - B. $C = \text{msg_time}$
 - C. $C = \max(C, C_{\text{msg}}) + 1$
 - D. $C = C_{\text{msg}} - 1$

70. For an internal event, a Lamport clock does:
- A. $C = C + 1$
 - B. $C = \max(C, C_msg)$
 - C. $C = 0$
 - D. Nothing
71. What is the key limitation of Lamport clocks?
- A. They violate causality
 - B. A smaller timestamp does NOT prove causation — they cannot detect concurrency
 - C. They require $O(n)$ space
 - D. They need a global clock
72. Lamport clocks guarantee which direction?
- A. $\text{clock}(A) < \text{clock}(B)$ implies $A \rightarrow B$
 - B. $A \rightarrow B$ implies $\text{clock}(A) < \text{clock}(B)$
 - C. Both directions
 - D. Neither direction
73. A vector clock gives each process:
- A. A single integer counter
 - B. A whole vector of counters — one slot per process
 - C. A physical clock
 - D. No state
74. On a vector-clock send, a process:
- A. Increments its own slot and attaches the whole vector
 - B. Resets all slots to zero
 - C. Only reads the vector
 - D. Takes the max with the message
75. On a vector-clock receive, a process:
- A. Sends a copy of its vector
 - B. Takes the component-wise max with the message, then bumps its own slot
 - C. Increments every slot equally
 - D. Does nothing
76. Vector clocks provide which complete property that Lamport clocks lack?
- A. $O(1)$ space
 - B. $A \rightarrow B$ if and only if $VC(A) < VC(B)$, enabling concurrency detection
 - C. Faster messages
 - D. Physical-time ordering

77. Two events are concurrent under vector clocks when:
- A. $VC(A) = VC(B)$
 - B. Neither $VC(A) \leq VC(B)$ nor $VC(B) \leq VC(A)$
 - C. $VC(A) < VC(B)$
 - D. One clock is zero
78. The space cost trade-off of vector clocks is:
- A. $O(1)$ per message
 - B. $O(n)$ space per message, where n is the number of processes
 - C. $O(n^2)$ per message
 - D. Zero
79. Which production systems are cited as using vector clocks / version vectors?
- A. Spanner and ZooKeeper
 - B. Dynamo, Riak, and CRDTs
 - C. HBase and Cassandra LWT
 - D. OpenMP and MPI
80. In the distributed Sleeping Barber, the bounded queue.Queue represents:
- A. The barber
 - B. The waiting room (fixed chairs)
 - C. The listening socket
 - D. A crashed node
81. In the TCP barber, the accept loop acts as the:
- A. Consumer
 - B. Producer that seats connections or rejects when full
 - C. Barber thread
 - D. Crash detector
82. In the barber code, queue.Full triggers which response to a client?
- A. SEATED
 - B. CUTTING
 - C. FULL, then the connection is closed
 - D. DONE
83. The barber's q.get() blocking on an empty queue is analogous to which Week-2 primitive?
- A. A mutex lock
 - B. `cond_var.wait()`
 - C. A semaphore release
 - D. A barrier

- 84.** What does TCP guarantee on a single link?
- A.** The whole network is reliable
 - B.** Ordered, reliable byte-stream delivery between two endpoints
 - C.** Global clock synchronization
 - D.** Byzantine fault tolerance
- 85.** The CAP theorem states a distributed data store can guarantee at most:
- A.** One of three properties
 - B.** Two of three properties at once
 - C.** All three always
 - D.** None during normal operation
- 86.** What do the three letters in CAP stand for?
- A.** Consistency, Availability, Partition tolerance
 - B.** Concurrency, Atomicity, Persistence
 - C.** Caching, Availability, Performance
 - D.** Consensus, Agreement, Partition
- 87.** Who first articulated CAP, and who formalized it?
- A.** Lamport (2000); Fischer (2002)
 - B.** Brewer (2000); Gilbert & Lynch (2002)
 - C.** Ongaro (2014); Ousterhout (2014)
 - D.** Deutsch (1994); Gosling (1996)
- 88.** In CAP, Consistency means:
- A.** Every request gets a non-error response
 - B.** Every read returns the most recent write, or an error
 - C.** The system tolerates message loss
 - D.** Replicas eventually converge
- 89.** Why can't you simply 'drop P' in CAP?
- A.** Partitions are chosen by the designer
 - B.** Real networks partition — it's a matter of when, not if
 - C.** P is the least important property
 - D.** Dropping P improves consistency
- 90.** The genuine CAP choice between C and A arises:
- A.** At all times
 - B.** Only during a partition
 - C.** Only when there is no partition
 - D.** Only at startup

- 91.** A CP system, during a partition, will:
- A.** Always respond, possibly with stale data
 - B.** Refuse to serve stale data, returning errors instead of wrong answers
 - C.** Drop partition tolerance
 - D.** Become unavailable permanently
- 92.** Which systems are given as examples of AP (choose Availability)?
- A.** HBase, ZooKeeper, Spanner
 - B.** Cassandra, DynamoDB, CouchDB
 - C.** Raft, Paxos, FLP
 - D.** Redis, etcd, Consul
- 93.** Which statement about CAP is a misreading?
- A.** The choice only appears during a partition
 - B.** 'Pick any 2 of 3, always' — true regardless of partitions
 - C.** PACELC extends CAP
 - D.** Partitions are imposed by the network
- 94.** From strongest to weakest, the consistency spectrum is:
- A.** Eventual, Causal, Sequential, Strong
 - B.** Strong, Sequential, Causal, Eventual
 - C.** Causal, Strong, Eventual, Sequential
 - D.** Sequential, Strong, Eventual, Causal
- 95.** Strong (linearizable) consistency makes the system behave as if:
- A.** Replicas may disagree indefinitely
 - B.** There is one single copy of the data, with operations atomic at one instant in real time
 - C.** Only causally related events are ordered
 - D.** No coordination is needed
- 96.** Causal consistency guarantees that:
- A.** All operations have one global real-time order
 - B.** Causally related operations are seen in the same order everywhere; concurrent ones may reorder
 - C.** Replicas never disagree
 - D.** Every read sees the latest write
- 97.** Eventual consistency promises that:
- A.** Reads always see the latest write
 - B.** Given no new updates, all replicas eventually converge to the same value
 - C.** Operations are atomic in real time
 - D.** No merge rule is needed

- 98.** Which consistency model has the highest coordination cost?
- A.** Eventual
 - B.** Causal
 - C.** Strong
 - D.** Sequential
- 99.** The eventual-consistency simulation converges using which merge rule?
- A.** First-writer-wins
 - B.** Last-writer-wins on the larger (version, value) pair — deterministic
 - C.** Random selection
 - D.** Majority vote
- 100.** Which three properties must any correct consensus protocol satisfy?
- A.** Consistency, Availability, Partition tolerance
 - B.** Agreement, Validity, Termination
 - C.** Safety, Liveness, Speed
 - D.** Crash, Omission, Byzantine